By TOUCH DEN · November 11, 2026

What an Operating System Does

Video 1 · The OS, the Kernel, and System Calls

How Computers Work · Section 5

In this video, you will learn:

- What an OS is, and why every computer needs one
- The kernel, and user space vs kernel space
- System calls

The OS as the manager

Between bare hardware and your apps sits the **operating system** (the middle layer).

It manages the hardware and shares it safely among all the programs that want it.

Like a kitchen manager: the hardware is the stoves and knives, the programs are cooks all wanting them at once. The manager keeps them out of each other's way.

A program can't touch the disk or network directly, **it asks the OS**. That funnel makes the OS the center of security.

The kernel, and the two spaces

The **kernel** is the core program, always running, with **full control** of the machine. The manager's brain and hands.

The CPU enforces a hard line:

- **Kernel space:** the kernel, full privilege, direct hardware access.
- **User space:** normal programs, limited, **no** direct hardware access.

A user-space app that crashes or misbehaves **can't** take down the machine. **Control the kernel → control everything** (the top prize for attackers).

System calls

If apps can't touch hardware, how does your editor save a file? It **asks the kernel**, through a **system call**.

A system call is the **controlled doorway** from user space into kernel space. Open a file, use the network, start a program → ask the kernel; it checks permission, does it, returns the result.

Why they matter for security:

- **Checkpoints:** every real action goes through one, where the kernel says yes or no.
- **Watch points:** tools observe a program's syscalls to see what it *really* does (malware analysis).

Recap: OS, kernel, syscalls

- The **OS** manages hardware and shares it; apps ask it for everything.
- The **kernel** is the fully privileged core; **user vs kernel space** is a CPU-enforced wall.
- **System calls** are the controlled doorway, the OS's checkpoint and a prime watch point.

Next video: multitasking, and the OS as the main security boundary.

What an Operating System Does

Video 2 · Multitasking and the Security Boundary

How Computers Work · Section 5

In this video, you will learn:

- Drivers and hardware
- Multitasking (context switches)
- Users, permissions, and privilege escalation

Drivers, and multitasking

Drivers: software that lets the kernel talk to specific hardware (a translator). Most run in **kernel space** with full privilege → a serious attack surface (a driver flaw = a kernel flaw).

Multitasking: a core runs one stream at a time, so the OS switches between programs thousands of times a second, a **context switch**, saving each program's state.

- Looks simultaneous; with several cores, some is truly parallel.
- Each program's state is **saved and isolated**, one can't read another's working data.

Users and permissions

The OS enforces **who may do what**, authentication and authorization, in practice.

- Every file and action has **permissions** tied to **users**.
- One all-powerful account: **root** / **superuser** (Linux, macOS) or **Administrator** (Windows).
- Normal accounts are deliberately **limited**, least privilege in action.

A limited account means malware running as you **can't** change the system, the damage is contained.

The OS is the main security boundary

Attackers work hard to **escalate privilege**, go from an ordinary account to **root / Administrator**.

- The powerful account can change the system and read any data.
- It's the **usual next step after breaking in** (a whole part of the pentest courses).

The OS isolates programs (user vs kernel space), controls every action (system calls), and enforces permissions. **Almost every attack is an attempt to break one of these OS boundaries.**

Recap: the security boundary

- **Drivers** are privileged kernel code, a real attack surface.
- **Multitasking** switches the CPU fast (context switches) and keeps programs isolated.
- The OS enforces **users and permissions**; attackers try to **escalate to root/Administrator**.
- The OS is **the main security boundary**, most attacks try to break it.

Next lesson: where programs actually live and run, processes and their memory.